Branched Multi-Turn GRPO for Code-Policy Generation on a Single GPU

Abstract

Multi-turn RL for code-generating agents fails mainly at credit assignment: a single trajectory-level advantage cannot tell whether success came from good planning or good final execution. This paper targets that gap in code-policy generation, where an LLM writes a Python `policy` and receives only the final scenario reward as supervision. The central idea is to treat chat turns as macro-steps: branch rollouts at turn boundaries, estimate per-turn Monte Carlo values, and assign turn-specific advantages (VinePPO-style, but at turn granularity). To make this practical on a single 24 GB GPU, loss computation is decomposed across turns using the additivity of the token-summed clipped surrogate, reducing peak LM-head activation memory from $\sum_n T_n$ to $\max_n T_n$ without changing the accumulated gradient. The resulting method is a compact synthesis of existing RL ingredients (GRPO, VinePPO, hierarchical credit assignment), specialized for multi-turn code-policy training under tight compute constraints. Experiments instantiate the method at $N{=}2$ with $G_1{=}G_2{=}4$; the extension to $N{>}2$ is straightforward.

1 Introduction

Reinforcement-learning fine-tuning of LLMs has converged on a small set of recipes: PPO with a learned critic, GRPO and its DAPO/CISPO variants which replace the critic with a group-relative Monte Carlo baseline, and process-supervision pipelines that train an auxiliary reward model. Most of this work targets single-turn reasoning, where the trajectory is a prompt followed by one completion and the reward is observed at the end. Extending these recipes to multi-turn agentic settings exposes a credit-assignment problem: a single trajectory-level advantage broadcast to every model token cannot distinguish a strong first-turn strategy followed by poor execution from the converse.

This problem is studied in the setting of *code-policy generation*: the LLM emits a Python `def policy(state)` body that is executed across environment scenarios in an isolated subprocess. The artifact is code; the LLM is removed from the inner control loop at deployment. The protocol uses N chat turns: intermediate turns may draft a candidate `policy` (whose scenario reward is returned as a tool message) or run a `main` scratchpad (stdout returned as feedback), and the final turn commits the scored `policy`. The scheme is instantiated and evaluated at $N{=}2$.

The training scheme has two components. First, trajectory-level credit assignment is replaced with a per-turn branched rollout and per-turn MC value estimates — the macro-step analog of VinePPO's intra-completion branching, with hierarchical baselines conditioned on the prefix at each turn boundary. Second, to make the resulting trajectories trainable on a 24 GB L4, the additivity of the token-summed clipped surrogate over disjoint token subsets sharing the same global normalizer is exploited to run forward and backward passes for each turn independently. Peak LM-head saved-activation memory drops from $O(B \cdot \sum_n T_n \cdot |V|)$ to $O(B \cdot \max_n T_n \cdot |V|)$ without changing the accumulated gradient: the parameter gradients summed across the per-turn passes equal the single-pass gradient exactly. The trade-off is compute — each later pass recomputes the prefix of earlier turns — not gradient quality.

This combination is a straightforward specialization of standard mechanisms rather than a new algorithm. The contribution is a clean assembly that fits on a single small GPU, plus a loss decomposition that makes that budget feasible.

2 Related work

Critic-free LLM RL. GRPO [1] and its token-summed variants (DAPO [2], CISPO) replace PPO's learned critic with a group-relative MC baseline. The token-summed form is used because its additivity over token subsets makes the per-turn loss decomposition exact.

Branching for credit assignment. The closest existing method is VinePPO [3], which estimates $\widehat{V}(s_t)$ at chosen intra-completion states via on-policy MC re-branching and uses it as a per-token baseline. The same mechanism is used, but branching occurs only at *turn boundaries* of a chat trajectory and broadcast the resulting advantage as a constant within each turn — group-relative within a turn, VinePPO-like across turns.

Hierarchical multi-turn agents. ArCHer [6] adopts the same macro-step decomposition (per-turn value, per-token policy update) but learns the per-turn value with a critic. Here it is estimated by MC over G_2 continuations, trading variance for the cost of fitting a value head. ReBN [5] replaces GAE with batch-normalized MC returns; the hierarchical mean used here can be viewed as ReBN partitioned by turn-1 prefix. RAGEN's StarPO [4] is the canonical flat-baseline critic-free MC scheme for multi-turn LLM RL; setting $G_2 = 1$ collapses our scheme to it.

$G_1 \times G_2$ tree rollouts. The same tree shape appears in process-reward-model training pipelines such as Math-Shepherd [7] and OmegaPRM [8], where the per-prefix MC mean labels intermediate steps for a learned reward model. The tree is used as a policy-gradient baseline rather than as a labelling oracle.

Code policies. Voyager [9] and FunSearch [10] also produce executable code as the artifact, but rely on frozen-LLM prompting plus skill-library accumulation or evolutionary mutation respectively. The LLM is adapted with gradient-based RL of LoRA weights and a scratchpad turn between prompt and committed code.

3 Method

3.1 Setup

The agent is a language model π_θ over chat-template trajectories. The base is `Qwen2.5-Coder-3B-Instruct` with frozen weights W; θ parameterizes a LoRA adapter $W' = W + AB^\top$. A task $\mathcal{T}$ specifies environment source code, S scenarios $\{\sigma_i\}$ that vary the seed and hidden parameters, and a per-scenario reward. The scoring oracle runs a candidate policy body in a sandboxed subprocess against each scenario and returns $R(\text{code}; \mathcal{T}) = \sum_i r_i(\text{code}; \sigma_i)$. Validation failures, errors, and timeouts on σ_i score the *error floor* $r_i = -200$, calibrated below any running-but-failing trajectory — otherwise errored rollouts ($r = 0$) outrank running-but-poor rollouts and the gradient rewards crashing.

3.2 Chat-template protocol

Each turn the model emits one fenced code block tagged `policy` or `main`. `policy` defines `def policy(state)`, which the tool executes across all scenarios and reports the summed reward as a user-role message. `main` is arbitrary scratchpad code with no environment access; the tool

returns truncated stdout. The final turn must be `policy`; `main` or untagged final turns are floored. Malformed intermediate turns are likewise floored and surfaced as a textual rejection. A length-N trajectory is $\tau = (s, y^{(1)}, m^{(1)}, \ldots, y^{(N-1)}, m^{(N-1)}, y^{(N)})$, with completions $y^{(n)}$ alternating with tool messages $m^{(n)}$, and $R(\tau) = R(\text{code in } y^{(N)})$ the only scalar reward. The remainder of the paper specializes to N=2: a single intermediate turn (draft policy or scratchpad) followed by a committed `policy`. Generalizing the rollout, advantages, and loss decomposition below to N>2 is straightforward (see §6).

3.3 Token-summed clipped surrogate

For trajectory b at position t, with importance ratio $\rho_t^b = \pi_\theta / q$ and per-token advantage A_t^b,

$$\mathcal{L}(\theta) = -\frac{1}{N_{\text{tok}}} \sum_{b,t} \mathbf{1}\{\text{model token}\} \cdot \min\big(\rho_t^b A_t^b,\ \text{clip}(\rho_t^b,\ 1-\epsilon_{\text{lo}},\ 1+\epsilon_{\text{hi}}) A_t^b\big), \tag{1}$$

with N_{tok} the count of model tokens in the global batch.[1] (1) is *additive over disjoint token subsets*: for any partition of model-token positions, the loss equals the sum of partial losses computed with the same N_{tok}. This is the property the per-turn decomposition (§3.6) exploits.

3.4 Branched rollout

For each prompt s, sample G_1 turn-1 variants in one `model.generate` call: $y_i^{(1)} \sim q(\cdot \mid s)$, $i = 1, \ldots, G_1$. For each $y_i^{(1)}$ execute the tool to obtain m_i, then sample G_2 continuations $y_{i,j}^{(2)} \sim q(\cdot \mid s, y_i^{(1)}, m_i)$ in one call. Score each leaf to obtain $\{R_{i,j}\}$. Generate-call cost per prompt is $1 + G_1$, versus $2G_1G_2$ for an unbranched rollout with the same number of leaves.

3.5 Per-turn MC value estimates and advantages

Two empirical values, both pure averages over the tree:

$$\widehat{V}_0 = \frac{1}{G_1 G_2} \sum_{i,j} R_{i,j}, \qquad \widehat{V}_1^{(i)} = \frac{1}{G_2} \sum_{j} R_{i,j}. \tag{2}$$

$\widehat{V}_0$ estimates the initial-state value; $\widehat{V}_1^{(i)}$ the value after committing to turn-1 variant $y_i^{(1)}$. Per-turn advantages, broadcast to every token of the corresponding turn:

$$A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0 \qquad \text{(over } y_i^{(1)}\text{)}, \tag{3}$$

$$A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)} \qquad \text{(over } y_{i,j}^{(2)}\text{)}. \tag{4}$$

The turn-2 advantage uses the conditional mean given the turn-1 prefix rather than the global per-prompt mean: a poor turn-2 sample drawn from a strong turn-1 is recognized as poor instead of inheriting its prefix's above-average outcome. Tool-message tokens are masked via `tool_mask`; their advantage is zero.

3.6 Per-turn loss decomposition

The token-summed clipped surrogate (1) factors over any partition of the model-token positions into disjoint subsets that share the same global normalizer N_{tok}. Letting $\mathcal{I}_n$ index the model tokens of turn n,

$$\mathcal{L} = \sum_{n=1}^{N} \mathcal{L}_{\mathcal{I}_n}, \qquad N_{\text{tok}} = \sum_{n=1}^{N} |\mathcal{I}_n|. \tag{5}$$

[1] This matches the DAPO and CISPO recipes; per-row averaging variants (original GRPO, BNPO, Dr.GRPO) divide by per-row token counts.

Per-row averaged variants (original GRPO, BNPO, Dr.GRPO) do *not* factor this way: each row carries its own denominator, so the partial sums do not combine into the single-row loss. The token-summed form is what makes the decomposition exact.

(5) is exploited to compute $\mathcal{L}$ in N forward/backward passes rather than one. Pass n forwards $[s \mid y^{(1)} \mid m^{(1)} \mid \ldots \mid y^{(n)}]$, computes $\mathcal{L}_{\mathcal{I}_n}$, backwards, and frees. By (5) the parameter gradients accumulated across the N passes equal the single-pass gradient exactly.

Why this saves memory. The dominant activation cost is the LM head's logit tensor, with $|V| \approx 152{,}000$ for our base model. The HF `logits_to_keep` API materializes logits only at the positions where they are needed for the loss (Appendix A); at training time those are the model-emitted token positions. In a single forward+backward over the full trajectory, the autograd graph holds the saved-for-backward LM-head tensors at all $\sum_n T_n$ model-token positions across all turns simultaneously, until backward completes. By computing each turn's loss in a separate forward+backward, only one turn's saved-for-backward tensors are held at peak ($T_n^{\max}$ positions), and the graph is freed before the next turn begins. The decomposition is gradient accumulation at the turn level within a trajectory: same accumulated gradient by (5), lower peak activation memory by a factor of roughly N when turns are equal-length. Appendix A has concrete numbers for N=2; at our settings the saving over a careful single-pass implementation is roughly 2×.

Restrictions. The decomposition's exactness has one *conceptual* requirement and two *implementation* requirements. Conceptual: the token-summed normalizer of (1) — per-row averaged variants do not factor across token-position partitions, as discussed above. Implementation: $\beta = 0$ (a KL term to π_{ref} would require per-segment slicing of reference log-probabilities) and `num_iterations`= 1 (so cached `old_per_token_logps` need no re-segmenting). The two implementation requirements can be lifted with extra bookkeeping at no theoretical cost (§6).

4 Experimental setup

Why this environment. Standard RL benchmarks are a poor fit for evaluating LoRA fine-tuning of a code-policy LLM. CartPole and similar two-line tasks are solved at iteration 0 by the base model, giving a flat learning curve. LunarLander, Pendulum, MountainCar, and other named environments are heavily represented in pretraining corpora, so any improvement is confounded with recall rather than learning. The task therefore requires source code that the model has to *read* rather than recognize, with non-trivial dynamics and a clear ceiling above the base-model prior. The puck-on-tilted-plane environment below was designed to fit those constraints: a small custom dynamics file, hidden per-scenario parameters that defeat look-up strategies, and a tight thrust envelope that rules out the naive one-line policy.

A second motivation, specific to the multi-turn protocol, is partial observability. The hidden tilt, drag, and thrust limit are not part of the state vector but they fully determine which control gains work; a policy that writes itself blind to those parameters is limited to whatever conservative gains succeed across the whole prior. The protocol gives the model an explicit affordance for *system-identification probes* in the rehearsal turn: a `main` scratchpad that applies short thrust patterns, inspects the resulting velocities via stdout, and uses the readings to size the committed policy's gains. Whether the trained model actually exploits this affordance — rather than collapsing to a robust prior — is itself an empirical question the protocol makes testable.

Environment. A 2D puck-on-tilted-plane controller. The puck moves in $[-6, 6]^2$ under a constant hidden tilt (t_x, t_y) and linear drag μ; the policy receives $\{p_x, p_y, v_x, v_y, g_x, g_y\}$ and must drive the puck to a goal within 120 steps. Hidden parameters — tilt, drag, thrust limit $a_{\max}$,

initial velocity — vary per scenario and are not in the state. The thrust envelope is set to $a_{\max} = \|\text{tilt}\|_2 \times 1.05 + \epsilon$ so a naive straight-at-goal policy fails because it cannot cancel tilt drift. Per-step reward is $-\Delta t \cdot \text{dist}(p, g)$ with a $+50$ bonus on reaching the goal; errored episodes hit the floor of §3.1.

Baseline gap. Two hand-written policies fix the headroom: naive aim ($\approx 26\%$ goal rate over 200 seeds) and PD-with-tilt-feedforward ($\approx 73\%$). The gap is the room LoRA fine-tuning has to exploit.

Model and compute. `Qwen2.5-Coder-3B-Instruct`, LoRA rank $r = 16$ on attention and MLP projections, on a single 24 GB L4 GPU. bf16 weights, gradient checkpointing on. The setup uses $G_1 = G_2 = 4$ (16 leaves per prompt) and a per-turn token cap of 400.

Training. The token-summed clipped surrogate (1) with $\epsilon_{\text{lo}} = 0.2$, asymmetric upper clip, $\beta = 0$, and the per-turn advantages of (3)–(4). No reference KL, no critic, no GAE, no context truncation across turns.

Diagnostics. Logged diagnostics include $\widehat{V}_0$ and the between-variant standard deviation of $\widehat{V}_1$, the within-variant standard deviation, and their ratio (the *signal ratio*); the per-turn advantage standard deviations; the fraction of trajectories at the error floor; and the count of `generate` calls per prompt (expected $1 + G_1$). The signal ratio guides allocation between G_1 and G_2 at fixed budget: > 1 means turn-1 variation dominates and a larger G_1 helps; < 1 means within-variant noise dominates and a larger G_2 helps.

5 Evaluating the per-turn credit assignment

The per-turn loss decomposition (§3.6) is purely a memory enabler — it does not change the gradient. The substantive claim that requires empirical support is the *credit-assignment* contribution: hierarchical per-turn advantages (3)–(4) on a $G_1 \times G_2$ tree do better than a single trajectory-level advantage broadcast across both turns. Table 1 defines a six-arm ablation, ordered from the weakest baseline to the proposed method. Adjacent rows isolate a single axis where possible (E vs. D isolates the tree; F vs. E isolates the per-turn baseline). The frozen-to-trained jump (B vs. C) and the single-to-multi-turn jump (C vs. D) each change two axes and are interpreted accordingly.

Primary metric. Held-out goal rate on a stress distribution (more extreme tilts/drag than train) at the best checkpoint, reported as median and IQR over ≥ 3 seeds per arm, plotted against gradient steps and against wall-clock. Statistical test: paired Wilcoxon over per-seed best-checkpoint goal rates; report effect size, not just p. The minimum convincing comparison is F > D at matched leaves.

Diagnostic checks. Two further measurements sharpen the mechanism story.

1. *Turn-1 quality curve.* Track the mean reward of turn-1 `policy` drafts (when turn-1 is in policy mode) as a function of training step. Under D the trajectory-level advantage projects onto turn-1 only through the global mean; under F, $A_i^{(1)}$ provides a direct gradient signal on turn-1 quality. F should show a steeper turn-1 reward rise than D.
2. *Signal-ratio conditioning.* The training loop already logs the per-prompt signal ratio $\widehat{\sigma}(\widehat{V}_1^{(i)})/\overline{\widehat{\sigma}(R_{i,\cdot})}$. Bucket prompts by their iteration-0 signal ratio. Ratio > 1 means turn-1 choice dominates the variance and per-turn credit should help; ratio < 1 means within-variant noise dominates and per-turn credit should not. F's advantage over D should grow

Arm	Rollout structure	Advantage	What it isolates over the previous arm
A — Frozen, one-shot	N=1 inference, no training	—	Base model's prior on the code-policy interface
B — Frozen, multi-turn prompted	N=2 inference, no training	—	Does the two-turn protocol help at inference time? (Voyager-/FunSearch-style frozen-LLM use)
C — Single-turn GRPO	G flat N=1 rollouts, trained	Standard GRPO advantage	Does RL fine-tuning help at all?
D — Flat StarPO	G independent N=2 trajectories, trained	$A(\tau) = R(\tau) - \bar{R}$, broadcast to all tokens	Does multi-turn training help over single-turn? (RAGEN-style)
E — Branched, flat	$G_1 \times G_2$ tree, trained	$A_{i,j} = R_{i,j} - \bar{R}$ on all tokens	Does the tree alone help (more leaves per generate call), independent of credit?
F — Branched, hierarchical (ours)	$G_1 \times G_2$ tree, trained	$A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0$ on $y_i^{(1)}$; $A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)}$ on $y_{i,j}^{(2)}$	The per-turn baseline contribution

Table 1: Ablation arms, ordered from weakest baseline (top) to ours (bottom). Trained arms (C–F) match leaf count $G = G_1 G_2 = 16$ so they see the same number of scored trajectories per step. D uses $2G$ `generate` calls per prompt; F uses $1 + G_1$. Report both equal-rollouts and equal-wall-clock comparisons. Frozen arms (A, B) use the same Qwen2.5-Coder-3B-Instruct base with no gradient updates, and serve as training-free references analogous to Voyager- or FunSearch-style frozen-LLM uses of the code-policy interface.

with the bucket's signal ratio. If it does not, the mechanism is not doing what theory predicts and the headline claim weakens.

Failure modes to declare upfront. If F $\not>$ D under matched leaves, the hierarchical baseline does not improve results in this domain and the contribution reduces to "branching amortizes generate calls" (E vs D). If F $\approx$ E but both beat D, the gain is from the tree structure, not from the per-turn baseline. If the trained arms (C–F) do not clearly beat the frozen arms (A, B), RL fine-tuning adds little over the base model's prior under this protocol, and the work is better framed as prompt-engineering rather than training. If B > A, the multi-turn protocol itself provides training-free gains, and that gap should be part of the headline result.

6 Limitations and future work

The scheme is instantiated and evaluated at N=2. Extension to N>2 is straightforward: each $\widehat{V}_n^{(\cdot)}$ becomes the conditional MC mean over the subtree below it, and the two-pass loss decomposition becomes an N-pass decomposition by the same additivity argument ((5)). Neither is conceptually new, but both add bookkeeping and are not implemented or benchmarked here. This work assumes deterministic environments given the seed — under stochastic dynamics the leaf rewards become noisier estimates of state values and G_2 would have to grow accordingly. The $\beta = 0$, `num_iterations`= 1 restrictions are implementation choices that the same decomposition argument can lift with extra per-segment slicing of reference log-probabilities and cached sampling log-probabilities.

Token subsampling. A direct extension is to backprop through only a fraction $\alpha \in (0, 1]$ of the model-emitted tokens per turn rather than all of them. Because each turn's advantage $A^{(n)}$

is a single scalar broadcast across all of that turn's tokens, every token contributes a sample of the same population gradient: subsampling is a sample-variance loss, not the information loss it would be in supervised fine-tuning where each token has a distinct target. The token-summed normalizer of (1) keeps the resulting estimator unbiased for any α when N_{tok} is updated to count only the kept positions; per-row averaged variants would be biased by uneven drops across trajectories, an additional reason the token-summed form is the right baseline. The memory benefit compounds with the per-turn decomposition: peak LM-head saved-activation memory drops from $\max_n T_n$ to $\alpha \max_n T_n$, on top of the $\sum_n T_n \to \max_n T_n$ saving of §3.6. Two variants suggest themselves as ablation arms. *Uniform per-turn:* drop a fixed fraction within every turn, to test whether the variance increase hurts convergence at all given the broadcast-advantage redundancy. *Asymmetric per-turn:* keep all of the committed turn $y^{(N)}$ but only α of the rehearsal turns $y^{(<N)}$, to test whether explicitly biasing learning toward the artifact helps over treating all turns symmetrically. The asymmetric variant has no single-turn analog. These experiments were not run here.

7 Conclusion

A critic-free, on-policy multi-turn RL scheme for code-policy generation that combines branched rollouts with per-turn MC value estimates and a per-turn loss decomposition. The algorithmic ingredients are individually standard; the assembly is small enough to train on a single 24 GB GPU, and the per-turn decomposition — which the token-summed normalizer makes exact — is what keeps it that way as the number of turns grows.

A Memory accounting and the trailing-window LM-head

This appendix walks through why a single forward over a multi-turn trajectory is expensive and where the per-turn decomposition's saving comes from. It is written for readers who are familiar with policy-gradient RL but not necessarily with the internals of LLM training. The argument turns on a single API constraint in HF `transformers` which we make explicit here rather than in the body.

The LLM as an autoregressive policy. A causal LLM is a stochastic policy whose action space at each step is the vocabulary $\mathcal{V} = \{1, \ldots, |V|\}$. Given a sequence of T token IDs as input, the model produces, at each position t, a logit vector $\ell_t \in \mathbb{R}^{|V|}$; the next-token distribution is $\text{softmax}(\ell_t)$, and the log-prob of having emitted token a at position $t{+}1$ is $\log \text{softmax}(\ell_t)_a$. To form the policy-gradient surrogate (1) we need exactly these log-probs at every position t such that $t{+}1$ was a *model-emitted* token. Logits at other positions — prompt positions, and (in our multi-turn setting) tool-message positions — are not needed for the gradient and are wasted compute if we materialize them.

Where the memory goes. The model splits into two pieces with very different memory profiles. We distinguish three buckets because the asymmetry between them is what motivates the decomposition: *parameter weights*, *per-position activations saved for backward*, and *per-position output of the LM head*.

Transformer body. A stack of $L \approx 36$ blocks (attention + MLP), holding the bulk of the $3{\cdot}10^9$ parameters. Each block produces per-position activations of width $O(d)$ with $d \approx 2048$. Without optimization the body's saved activations for backward are $O(L \cdot d)$ per position. We use gradient checkpointing, which recomputes intermediate block activations during backward instead of storing them, leaving only $O(d)$ per position saved at checkpoint boundaries. So in our setup the body holds many parameters but small per-position activations.

LM head. A single linear layer of shape $(d, |V|)$: $\sim 3{\cdot}10^8$ parameters — modest — but its per-position *output* is the full logit vector of width $|V| \approx 152{,}000$. The head sits at the end of the graph, so checkpointing it would buy nothing (there is nothing upstream to recompute through the loss): its full $(B, T, |V|)$ output is held until backward, with an equal-sized autograd buffer for the gradient.

The asymmetry. Once gradient checkpointing is on, the body keeps roughly $d \approx 2{,}000$ floats of activation per position, while the LM head keeps roughly $|V| \approx 152{,}000$. The head's per-position activation is $\sim 75\times$ the body's, and unlike the body it cannot be checkpointed away. Body parameter weights are larger in absolute terms but they are loaded once and shared across all positions in the batch; the head's $(B, T, |V|)$ output scales with both batch size and trajectory length. At our trajectory lengths the LM-head activation tensors are the term that pushes a single-pass forward over budget, while parameters, optimizer state, and gradient-checkpointed body activations are fixed costs that fit under any of the regimes below. *The decomposition therefore saves peak LM-head activation memory specifically, not total GPU memory in equal proportion* — an important distinction when reading Table 2.

The `logits_to_keep` parameter. Because most positions' logits are not needed for the loss, HF `transformers` lets you specify which ones to compute. The forward call accepts an argument `logits_to_keep` (historically `num_logits_to_keep`) of type `int` *or* 1D `torch.Tensor`.[2] With an integer K, the LM head is applied to the last K hidden states, producing logits of shape $(B, K, |V|)$. With a 1D tensor of indices, the head is applied at exactly those positions in the sequence dimension. Either way, only the selected positions' logits are materialized and held for backward. The transformer body still runs over all T positions because causal self-attention chains every position to its prefix, but the body is two orders of magnitude cheaper per position than the LM head.

The constraint that actually matters. The API can select arbitrary subsets of positions, so the issue is not which positions can be reached. The issue is autograd: a single forward+backward holds the saved-for-backward LM-head tensors at *all* selected positions in memory simultaneously, until `loss.backward()` returns. Peak LM-head activation memory therefore scales with the total number of selected positions, regardless of whether they are contiguous.

For multi-turn training the total is $\sum_n T_n$ — every model- emitted token across every turn. The per-turn decomposition sidesteps this by doing one forward+backward per turn and freeing the graph between, so peak LM-head memory scales with $\max_n T_n$ instead. The trick is identical in spirit to gradient accumulation across micro-batches: the same accumulated gradient is computed, but peak activation memory is shrunk by serializing what would otherwise be parallel.

Three regimes. Concretely, with $T_1 = T_2 = 400$, prompt ≈ 500, tool message ≈ 50, batch $B = 16$ leaves per prompt, $|V| = 152{,}000$, and 4-byte logits (the autograd intermediate is held in fp32 for the cross-entropy reduction even when weights are bf16):

The relevant comparison for this setting is (b) vs. (c) — a roughly $2\times$ saving for $N{=}2$, scaling to $\sim N\times$ as the number of equal-length turns grows. Regime (a) is shown for completeness but is not what a careful implementation actually does.

Arithmetic, not measured. The numbers in Table 2 are arithmetic from the LM-head output tensor shape $(B, K, |V|)$ at 4 bytes per element; peak end-to-end GPU memory under the three regimes has not yet been measured. End-to-end memory also includes parameter weights and

[2] See e.g. the Qwen2 forward signature in HF `transformers`: the slice is `slice(-K, None)` for an int and the tensor itself for a 1D `torch.Tensor`.

Regime	Positions through LM head	Logit tensor
(a) Single pass, no `logits_to_keep`	$T_{\text{full}} = 1350$	$\approx 13.1\,\text{GB}$
(b) Single pass, tensor `logits_to_keep` (model tokens)	$\sum_n T_n = 800$	$\approx 7.8\,\text{GB}$
(c) Two passes (ours), int `logits_to_keep`= $\max_n T_n$ per pass	400 per pass, freed between	$\approx 3.9\,\text{GB}$

Table 2: LM-head logit tensor size under each regime, plus an equal-sized autograd buffer for the backward pass (not shown). Regime (a) is the truly naive single forward. Regime (b) is what a careful single-forward implementation does, using the 1D-tensor form of `logits_to_keep` to select exactly the model-token positions across all turns. Regime (c) is the per-turn decomposition of §3.6, which holds at most one turn's LM-head saved-for-backward tensors at peak.

gradients, optimizer state, gradient-checkpointed body activations, KV-cache (during rollout, not training), masks, and allocator overhead. The decomposition's expected effect is on the LM-head activation term specifically, and a benchmark comparing measured peak GPU memory under (b) and (c) at the same batch size, trajectory length, and model would provide a direct empirical complement to the arithmetic above. That benchmark has not yet been run.

What the per-turn decomposition is and isn't. It is not a new memory optimization over `logits_to_keep`; it is gradient accumulation at the turn level within a trajectory. The HF API already supports specifying any subset of positions for which to compute logits, so all three regimes above use the same primitive — the difference is whether saved-for-backward tensors are held simultaneously through one backward call or sequentially through N backward calls. The non-trivial observation is that the gradients accumulated across per-turn passes equal the single-pass gradient *exactly* (eq. 5), and that this exactness depends on the token-summed normalizer of (1). Per-row- averaged GRPO variants would have to fall back to a single forward (regime b) or accept a normalizer approximation.

Implementation note. Pass 1 forwards $[s \mid y^{(1)}]$ as a self-contained sequence with `logits_to_keep` $= T_1$. Pass 2 forwards the full trajectory but packs $[s \mid y^{(1)} \mid m]$ into the prompt slot of HF's training interface and $y^{(2)}$ alone into the completion slot, so `logits_to_keep` $= T_2$ behaves as expected.

References

[1] Z. Shao et al. *DeepSeekMath*. arXiv:2402.03300, 2024.

[2] Q. Yu et al. *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. arXiv:2503.14476, 2025.

[3] A. Kazemnejad et al. *VinePPO: Unlocking RL Potential For LLM Reasoning Through Refined Credit Assignment*. arXiv:2410.01679, 2024.

[4] Z. Wang et al. *RAGEN: Training Agents by Reinforcing Reasoning*. 2025.

[5] Z. Liu et al. *GEM: Reinforcement Learning for Generalist Models*. 2025.

[6] Y. Zhou et al. *ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL*. ICML, 2024.

[7] P. Wang et al. *Math-Shepherd*. ACL, 2024.

[8] L. Luo et al. *Improve Mathematical Reasoning in Language Models by Automated Process Supervision*. arXiv:2406.06592, 2024.

[9] G. Wang et al. *Voyager*. TMLR, 2024.

[10] B. Romera-Paredes et al. *Mathematical discoveries from program search with large language models*. Nature, 2024.